

Od transkripta do CRM-ja

Zasnova avtomatizacije, ki iz zapisnika sestanka naredi povzetek, naloge, vpis v poslovni sistem in osnutek follow-up sporočila, ne da bi si pri tem karkoli izmislila.

Benjamin Salkič september 2026

KODA PROTOTIPA github.com/BenjaminSalkic/ai-zapisniki-sestankov

Izhodišče

Pri tej nalogi ni težko nič od tistega, kar svetovalec počne ročno. Prebrati transkript, izluščiti zahteve, napisati povzetek, vnesti naloge — vse to model naredi v enem klicu in bo večinoma naredil dobro. Težko je presoditi, ali je izluščeno res tisto, kar je bilo dogovorjeno.

Zato cilj ni, da AI naredi vse, ampak da naredi devetdeset odstotkov, človek pa v minuti potrdi ali popravi. Sistem, ki v petindevetdesetih odstotkih deluje, v petih pa tiho zapiše napačen rok, je slabši od ročnega dela, ker nihče ne ve, katerih pet odstotkov je. Ljudje mu nehajo zaupati hitreje, kot se izboljša, in potem preverjajo vse.

Zapis v sistem in vsak stik s stranko gresta skozi človeka.
Napačen notranji povzetek nekdo popravi, napačna e-pošta stranki je že zunaj.

Vsak podatek nosi dokaz.
Za vsako trditev mora model vrniti dobesedni citat iz transkripta. Citat preverimo s kodo, ne z drugim modelom.

Vprašanje 1 Potek od začetka do konca

Potek

- 1

Prožilec

Orodje za zapisovanje preko webhooka javi, da je transkript pripravljen. Od tu naprej vse teče asinhrono v vrsti.
- 2

Priprava konteksta

Datum, udeležence in ID stranke pobremo iz koledarja in CRM-ja, ne iz besedila. Stranko ujamemo po e-poštni domeni udeležencev, tako da model nikoli ne odloča, v čigav zapis pišemo.
- 3

Ekstrakcija

En klic modela z vsiljeno JSON shemo: povzetek, zahteve, odločitve, naloge z lastniki in roki, tveganja, komercialni signali, osnutek e-pošte in seznam nejasnosti.
- 4

Kontrole

Sloj navadne kode brez AI-ja preveri citate, roke, lastnike in pragove. Vsaka ugotovitev dobi stopnjo BLOCK REVIEW INFO. Če kaj pade, v CRM ne gre nič.

5

Potrditev

Svetovallec dobi v Slacku izpolnjen paket z označenimi spornimi mesti in tremi gumbi. Računam na pol minute dela; če bo trajalo dlje, se tega ljudje ne bodo držali.

6

Zapis

Zapis o sestanku in naloge gredo v CRM preko API-ja, follow-up pa se shrani kot osnutek. Pošlje ga človek, ker je e-pošta stranki edina stvar v verigi, ki je ni mogoče vzeti nazaj.

Vprašanje 2 Orodja in razlogi

Orodja

Transkripcija	Obstoječe orodje za sestanke (Teams, Zoom, Fireflies). Lasten Whisper samo, če posnetki ne smejo iz hiše.
Orkestracija	n8n, self-hosted. Vizualen potek, vgrajeni webhooki in retry, predvsem pa lahko kasneje kdo, ki ni razvijalec, zamenja kanal ali doda polje. Za popoln nadzor bi vzel Python in Temporal, a bi do uporabne verzije prišel počasneje.
Model	Claude Opus 5. Structured outputs, spodobna slovenščina, dovolj konteksta za cel transkript brez rezanja. Ob strošku bi rutinske sestanke pognal na Sonnetu.
Kontrole	Navaden Python. Ali se citat pojavi v besedilu in ali je datum za datumom sestanka, sta vprašanji z enim pravilnim odgovorom. Za to ne rabim modela.
Potrditev, stanje	Slack, ker so svetovalci tam in nova aplikacija pomeni novo navado, ki je ni. Stanje v Postgresu: vsak sestanek ima status, sicer ne veš, kaj visi, in ne moreš ničesar pognati še enkrat.

n8n in Python se ne podvajata. n8n je vodovod (webhook, kontekst, vrsta, Slack, čakanje na potrditev), Python je logika (shema, prompt, kontrole, razrešitev ID-jev, idempotenca, outbox). V produkciji je paket izpostavljen kot `POST /extract` in `POST /commit`, ki ju n8n pokliče. Meja teče tam, kjer teče razlika v tem, kako se stvari spreminjajo: prompt in kontrole je treba verzionirati, testirati in pregledati, vodovod pa se spreminja pogosto in ne bi smel zahtevati razvijalca.

Vprašanje 3 Kako AI pridobi strukturo

Ekstrakcija in shema

Namesto da bi model prosil za JSON in upal, mu shemo vsilimo preko API-ja (*structured outputs*). Odgovor je s tem zagotovo veljaven JSON s pravimi polji in tipi, kar odpravi cel razred sitnosti: manjkajoča polja, JSON zavit v markdown, niz namesto števila.

```
response = client.messages.parse(model="claude-opus-5", system=SYSTEM_PROMPT,
    messages=[{"role": "user", "content": user_prompt}],
    output_format=MeetingRecord) # Pydantic shema -> validiran objekt, ne niz
```

Prompt je zato krajši, kot sem pričakoval. Skrbi predvsem za to, da model ne zapolnjuje praznin:

Tvoja naloga je **ekstrakcija, ne interpretacija**. Uporablaj izključno informacije iz transkripta. Če podatka ni, vrni `null`: **prazno polje je pravičen odgovor, izmišljen podatek je najhujša napaka**. Vsako trditev podpri z `evidence.quote`, dobesednim odlomkom, kopiranim znak za znak. Odgovorno osebo navedi samo, če je bila izrecno določena. Roke pretvori v absolutne datume; ohlapnih ne ugibaj, pusti `null` in dodaj opombo v `unclear_points`.

Zadnji dve pravili sta nastali šele v drugem poskusu. V prvem osnutku je pisalo »če nisi prepričan, podaj najboljšo oceno«, kar je tu natanko narobe: model, ki ocenjuje, bo ocenil tudi rok, ki ga nihče ni izrekel.

```
"action_items": [{ "title": "Izvesti pilot in predstaviti rezultate",
    "owner_name": "Marko Zupan", "due_date": "2026-09-30", "priority": "high",
    "evidence": { "quote": "Rezultate pilota lahko predstavim do konca septembra." } }],
"unclear_points": ["Rok za predlog pogodbe ni bil dogovorjen."], "overall_confidence": 0.89
```

Polji `evidence` in `unclear_points` nista tam zaradi lepšega: prvo uporabi naslednji korak, drugo pride svetovalcu pred oči.

Vprašanje 4 Zapis v poslovni sistem

Zapis preko API-ja

Po potrditvi gresta ven `POST /meeting-notes` in `POST /tasks` za vsako nalogo. Bolj kot to, kaj v njiju je, se mi zdi pomembno, česa ni:

- **Model ne napiše nobenega ID-ja.** Vrne ime, sistem ga poišče v kadrovskem imeniku in šele takrat nastane `owner_id`. Isto velja za ID stranke in šifrante. Če imena v imeniku ni, naloga gre na pregled s predlogom najbližjega ujemanja.
- **Idempotenčni ključ** je hash sestanka, tipa zapisa in vsebine. Brez tega je vsak timeout potencialna podvojena naloga, in timeouti so tisto, kar se v praksi dogaja najpogosteje.
- **Sledljivost:** vsak zapis nosi sha256 transkripta, verzijo prompta, ime modela in e-pošto potrjevalca. Čez tri mesece bo to edini način, da komu odgovoriš, od kod je prišla neka naloga.

Vprašanje 5 Kako preprečimo izmišljene podatke

Kontrole

Nasvet »dodaj človeka v zanko« drži, sam po sebi pa ne zadošča. Če človeku pokažeš lep povzetek s petnajstimi postavkami, jih bo po tretjem sestanku potrdil, ne da bi jih prebral. Sistem mora sam povedati, kateri postavki ne zaupa. Pet plasti, od najcenejše proti najdražji:

Schema	Structured outputs: napačna oblika, manjkajoča polja in izmišljene vrednosti šifrantov so po konstrukciji nemogoči.
Citati	Vsak <code>evidence.quote</code> mora biti v transkriptu, normalizirano ujemanje s pragom 0,90.
Poslovna pravila	Lastnik obstaja v imeniku, rok ni pred sestankom in ne več kot leto naprej, komercialni podatek ima citat.
Pragovi	<code>confidence < 0,75</code> , naloga brez lastnika ali roka in vsak komercialni podatek gredo obvezno na pregled.
Potrditev	Obvezna pred zapisom v CRM in pred vsakim stikom s stranko.

Jedro je druga vrstica. Model, ki si izmisli nalogo, si mora izmisliti tudi citat, ta pa pade na navadnem iskanju po besedilu. Kontrola ne stane nič in ne uporablja AI-ja: idejo, da bi halucinacije lovil z drugim klicem modela, sem zavrgel, ker problem samo premakne za korak naprej. V prototipu sem v izvleček podtaknil nalogo, ki je v transkriptu ni, podpis pogodbe za 45.000 evrov. Kontrole jo ujamejo štirikrat:

```
[BLOCK ] action_items[4]: Citata ni v transkriptu (možna halucinacija): 'Strinjamo se s ceno...'
[BLOCK ] action_items[4]: Odgovorne osebe 'Janez Kranjc' ni v imeniku zaposlenih.
[BLOCK ] action_items[4]: Rok 2026-09-01 je pred datumom sestanka 2026-09-08.
[BLOCK ] commercial: Citata ni v transkriptu (možna halucinacija): 'Celotna implementacija...'
=== Status: blocked ===
```

Kar tu manjka in bi bilo prvo naslednje: eval set 30–50 resničnih transkriptov z ročno označenimi izhodi, ki teče ob vsaki spremembi prompta. Brez tega je vsako popravljanje prompta ugibanje, ki se sliši prepričljivo. Zanimata me dve številki: koliko pravih nalog sistem zgreši in koliko izmišljenih spusti skozi. Druga je dražja, ker prvo človek opazi sam.

Vprašanje 6 Kaj, če kakšen sistem ne dela

Odpornost na izpade

Vse stoji na eni odločitvi: transkript je izvorna resnica, vsak korak pa se da ponoviti iz shranjenega stanja. Dokler to drži, noben izpad ne pomeni, da mora kdo karkoli delati še enkrat ročno.

- **Model (API):** SDK ponovi ob 429 in 5xx. Ob daljšem izpadu gre sestanek v vrsto; zamuda ni škoda, svetovalec mora le vedeti, da zapisnik še ni pripravljen.
- **CRM API:** pet poskusov z eksponentnim odlogom in jitterjem, samo za napake, ki so lahko prehodne. Če tudi zadnji pade, gre potrjen zapis v **outbox** na disk in se odda, ko je CRM spet dosegljiv. Idempotenčni ključji poskrbijo, da se nič ne podvoji.
- **CRM zavrne (4xx):** ponavljanje se ne bo izšlo, zato gre zapis v dead-letter in svetovalcu, z navedenim razlogom.
- **Slack:** nadomestni kanal je e-pošta; sestanek ostane `awaiting_approval`, dokler ga kdo ne obdela.

Preizkusil sem primer s CRM-jem, ker je edini, ki sem ga znal verodostojno simulirati: ob ugasnjenem strežniku pipeline odloži tri zapise v outbox, po ponovnem zagonu s 40 % namerno vrnjenih napak pa jih odda vse in nobenega ne podvoji.

Vsak sestanek ima tudi status, od `extracted` do `written`, `blocked` ali `partially_written`, tako da se da vprašati, kaj visi in zakaj. Najpogostejša odpoved takih sistemov po mojem sploh ni tehnična, ampak ta, da nekdo pozabi klikniti *Potrdi*; zato sestanek, ki čaka več kot 48 ur, sproži opomnik.

Prototip in kaj bi bilo naslednje

Delujoč prototip osrednjega dela je v repozitoriju zgoraj, ker se o kontrolah proti halucinacijam lažje pogovarjamo, če jih je mogoče pognati. Python z eno zunanjo odvisnostjo, vključno z lažnim CRM-jem, ki zna simulirati izpade. Pognal sem tri scenarije: normalen potek (ponoven vpis ne ustvari dvojnikov), halucinacijo (štiri **BLOCK**, v CRM ne gre nič) in izpad CRM-ja (outbox, nato vse oddano brez podvojitev). Klica na pravi API nisem pognal, ker nisem imel ključa; koda sledi dokumentirani uporabi SDK-ja, vse ostalo teče na shranjenem odgovoru modela.

Naslednji koraki po vrsti: eval set na resničnih transkriptih, ker je brez meritve vse ostalo mnenje; dva tedna v načinu, kjer AI predlaga in človek vedno potrdi, z merjenjem, koliko popravkov je v resnici potrebnih; in šele če je ta delež nizek, samodejna potrditev za nizko tvegane dele. Komercialni podatki in komunikacija s stranko ostanejo pri človeku tudi potem.

Opomba o uporabi AI orodij. Nalogo sem delal s Claude Code, največ pri pisanju prototipa, se pravi tam, kjer sem vnaprej vedel, kaj hočem. Zasnovo sem določil sam, predvsem del s citati in ločnico med tem, kar sme napisati model, in tem, kar mora razrešiti sistem. Kjer je bilo treba presojati, prvega predloga nisem vzel: prvi prompt je bil preveč popustljiv, idejo o preverjanju halucinacij z drugim klicem modela pa sem zavrgel in jo nadomestil z navadno primerjavo besedila.